Applications of Zig Zag Functions in Quantum Algorithms

M.Chowdhury Written 12th October 2026* (Dates of server time stamp)* Draft version

Abstract

We encode the Sieve of Eratosthenes into the base-b digits of a single rational number built from geometric series. Two variants are given: a *counting* encoding, in which digit k is the number of non-trivial divisors of k up to a sieve bound U (or a weighted sum of them), and a *bitwise-OR* encoding, in which digit k is the compositeness indicator and which works in any binary base $b = 2^r$, including $b = 2$. A marker term flags 1 as non-prime. We prove exact digit formulas, carry-free base conditions, a digit-extraction identity and reversible circuits for primality, divisor-count and divisor-sum oracles; with quantum counting these estimate $\pi(N)$ to relative error ε in $O(\sqrt{\ln N}/\varepsilon)$ queries. Using the divisor-sum digit we give an explicit quantum *existence* search for odd perfect numbers. We study the transfer of the Zig–Zag construction from Pisano periods to sieve sequences: the counting sequence has minimal period $\mathrm{lcm}(2, \ldots, U)$ and the OR sequence the primorial; every digit oracle depends only on a gcd, so no family of digit oracles is injective on a period and the direct analogue of the Zig–Zag injectivity conjecture is false; stacking *shifted* copies in disjoint blocks repairs injectivity but recovers a quantity determined by U, so it is not a competitive prime-counting method (since the contribution is conceptual and not intended to be competitive with the algorithm in our paper). Appendices extend the encoding to truncated Dirichlet convolutions and their Boolean analogues, to higher-order (polynomial-weight) series, and to prime constellations, all unified as generalized Lambert transforms with a kernel $g_,$: a shift-and-NOR oracle supports a quantum existence and counting procedure for gathering *numerical evidence* on twin primes (evidence only; it cannot prove infinitude, we make no claim that it is the fastest method, and its advantage over classical sampling is a quadratic reduction in oracle queries) we also give a related method for prime constellations which is quadratically faster than the best known classical algorithm finding existnce of a prime constellations on an arbitrarty sized interval. Finally we show that quantum counting can find L from the L-th prime using $O(N)$ queries.

1 Introduction

The Sieve of Eratosthenes is usually expressed as a table from which multiples are struck. Here we store sieve information in the *digits of one number*: each base-b digit is an independent counter (or, in the OR variant, an independent flag). The encoding suits quantum computation because a digit can be read out reversibly by modular arithmetic, giving oracles to which amplitude estimation and amplitude amplification apply. The paper assumes an ideal universal gate model qunatum computer which is noise free. This paper also assumes the widely used conjecture that an arbitrary quantum circuit cannot be simulated in polynomial time on a classical only machine.

Contributions.

1. Exact digit formulas for the counting and bitwise-OR encodings, a marker for 1, carry-free conditions, a digit-extraction identity and reversible circuits (Sections 2–3).

2. Quantum counting of primes with explicit error and cost (Theorem 9).

3. A quantum existence search for odd perfect numbers built on the divisor-sum digit (Section 4).

4. The exact periods of both digit sequences, a no-go theorem for digit-oracle Zig-Zag injectivity and an injective repair by shifted stacking (Sections 5–7).

5. A lower bound showing that counting is an $O(N)$ method,not an $O(\sqrt{N})$ method, for $P_L \mapsto L$ (Section 8).

6. Appendices: Dirichlet-convolution and Boolean encodings (A, B), their higher-order versions (C, D) prime-constellation oracles with a twin-prime evidence procedure (E), a unifying generalized Lambert-transform framework (F) and a record of its Dirichlet-ring structure (G).

We do *not* claim an asymptotic improvement over the best known prime-counting algorithms.

Related work. Quantum counting of primes with a primality-test oracle was proposed by LS in [12]. Classically, $\pi(x)$ is computed in $\tilde{O}(x^{2/3})$ time by the Meissel–Lehmer family [10, 6] and in $O(x^{1/2+\epsilon})$ time by the Lagarias-Odlyzko analytic method [11], rigorously implemented by Platt [18]. Grover-type searches over explicit number-theoretic predicates exist for other problems, e.g. Goldbach partitions [13]. The Zig–Zag construction for Pisano periods is from the companion manuscript [17].

Conventions. $N = 2^\alpha$; $[P] \in \{0, 1\}$ is the truth value of P; $L_U := \operatorname{lcm}(2, \ldots, U) = \prod_{p \le U} p^{\lfloor \log_p U \rfloor}$; $U\# := \prod_{p \le U} p$; $\tau, \sigma, \omega, \mu, \varphi$ are the usual arithmetic functions. The *oracle function* $\mathrm{Oracle}_{\mathrm{type},U}(k)$ (Table 1) is the value that the quantum circuit computes in superposition, $|k\rangle|0\rangle \mapsto |k\rangle|\mathrm{Oracle}_{\mathrm{type},U}(k)\rangle$; the corresponding reversible unitary is written $\mathbf{O}_{\mathrm{type}}$. The symbols S_n, Σ, Ξ, Θ denote the *real numbers* (digit streams) whose digits the oracle functions return; they are not oracles themselves.

Error and repetition. All quantum algorithms here are bounded-error. A constant success probability (or $\Omega(1/\log\log L)$ for period finding) is amplified to $1-\delta$ by $O(\log 1/\delta)$ (respectively $O(\log\log L \cdot \log 1/\delta)$) independent repetitions with a classical check or a median. This multiplies the per-run costs stated below by a polynomial or polylogarithmic factor that does not change the asymptotic classification, and is absorbed in $\tilde{O}$. The oracles are implemented using Bennett reversility construction which transforms a classical funtion into a reversible qunatum function using garbage.

2 The digit-encoded sieve

Definition 1 *Let $U \ge 2$, $b \ge 2$ be integers and $w = (w_n)_{n=2}^U$ non-negative integers. Put*

$$S_n(b) = \frac{b^{-2n}}{1 - b^{-n}} = \sum_{j \ge 2} b^{-jn}, \qquad \Sigma_{U,b}^w = \sum_{n=2}^{U} w_n S_n(b), \qquad \Sigma_{U,b} := \Sigma_{U,b}^{\mathbf{1}}, \tag{1}$$

Concept	Oracle function (computed in superposition)	Reversible unitary
Unmarked counting digit (Thm 2)	$\mathrm{Oracle}_{\mathrm{count},U}(k)$	$\mathbf{O}_{\mathrm{count}}$
Marked counting digit	$\mathrm{Oracle}_{\mathrm{markedCount},U}(k)$	$\mathbf{O}_{\mathrm{markedCount}}$
Bitwise-OR / compositeness flag (Thm 6)	$\mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k)$	$\mathbf{O}_{\mathrm{bitwiseOR}}$
Divisor-sum digit (Section 4)	$\mathrm{Oracle}_{\mathrm{divisorSum},U}(k)$	$\mathbf{O}_{\mathrm{divisorSum}}$
Paired divisor sum (circuit, Section 4)	$\mathrm{Oracle}_{\mathrm{pairedDivisorSum}}(k)$	$\mathbf{O}_{\mathrm{pairedDivisorSum}}$
General weighted digit (weights w)	$\mathrm{Oracle}_{w,U}(k)$	–
Primality predicate	$\mathrm{Oracle}_{\mathrm{prime},U}(m)$	$\mathbf{O}_{\mathrm{prime}}$
Odd perfect number predicate	$\mathrm{Oracle}_{\mathrm{oddPerfect},U}(k)$	$\mathbf{O}_{\mathrm{oddPerfect}}$
Subset code (weights 2^n)	$\mathrm{Oracle}_{\mathrm{subsetCode},U}(k)$	–
Shifted stacking map (Thm 20)	$\mathrm{Oracle}_{\mathrm{stacked},U}(k)$	$\mathbf{O}_{\mathrm{stacked}}$
OR window map (Section 5.1)	$\mathrm{Oracle}_{\mathrm{windowOR},U,S}(k)$	$\mathbf{O}_{\mathrm{windowOR}}$
Constellation predicate (Appendix E)	$\mathrm{Oracle}_{\mathrm{constellation},U,C}(k)$	$\mathbf{O}_{\mathrm{constellation}}$
Boolean divisor transforms (Appendix B)	$\mathrm{Oracle}_{\mathrm{divisorOR},P,U}(k)$, $\mathrm{Oracle}_{\mathrm{divisorAND},P,U}(k)$, $\mathrm{Oracle}_{\mathrm{divisorXOR},P,U}(k)$	–
Higher-order digit (Appendix C)	$\mathrm{Oracle}^{(m)}_{f,U}(k)$	$\mathbf{O}^{(m)}_{f}$

Table 1: Notation: the oracle function is the value $|k\rangle|0\rangle \mapsto |k\rangle|\mathrm{Oracle}(k)\rangle$ computed coherently in superposition; the reversible unitary is the corresponding gate.

$W_U := \sum_{n=2}^{U} w_n$ *(*$= U-1$ *for* $w \equiv 1$*), and the* marked *series* $\Xi_{U,b} := b^{-1} + \Sigma_{U,b}$.

The first non-zero digit contributed by n sits at position $2n$, not n; this excludes the trivial divisor k itself. The marker b^{-1} puts a digit 1 at position 1. In base 2 it equals $0.5 = 0.1_2$, which is why 0.5 works in the binary case; in other bases 0.5 is not a single digit 1 at position 1, and b^{-1} is the correct general term.

Theorem 2 (Digit sieve) *If* $b > W_U$ *then* $\Sigma^{w}_{U,b} = \sum_{k\geq 1} \mathrm{Oracle}_{w,U}(k)\, b^{-k}$ *is the base-b expansion, with digits*

$$\mathrm{Oracle}_{w,U}(k) = \sum_{n\mid k,\ 2\leq n\leq U,\ n<k} w_n. \tag{2}$$

In particular $\mathrm{Oracle}_{\mathrm{count},U}(k) := \mathrm{Oracle}_{\mathbf{1},U}(k)$ *is the number of non-trivial divisors of* k *that are at most* U*, and the digits of* $\Xi_{U,b}$ *(for* $b \geq U$*) are* $\mathrm{Oracle}_{\mathrm{markedCount},U}(k) = \mathrm{Oracle}_{\mathrm{count},U}(k) + [k = 1]$.

Proof. The coefficient of b^{-k} is $\sum w_n$ over pairs (n, j) with $j \geq 2$, $jn = k$, i.e. $n \mid k$ with $k/n \geq 2$, i.e. $n < k$. Each coefficient is at most $W_U < b$, so no carries occur. The digits are not eventually $b-1$, since $\mathrm{Oracle}_{w,U}(k) = 0$ for every prime $k > U$. No $n \geq 2$ divides 1 properly, so the marker does not interact with the other terms. ■

Corollary 3 (Primality) *Let* $b \geq U$ *and* $1 \leq m < (U+1)^2$*. Then* $\mathrm{Oracle}_{\mathrm{markedCount},U}(m) = 0$ *iff* m *is prime.*

k	1	2	3	4	5	6	7	8	9	10	11	12
$\text{Oracle}_{\text{count},5}(k)$	0	0	0	1	0	2	0	2	1	2	0	3
$\text{Oracle}_{\text{markedCount},5}(k)$	1	0	0	1	0	2	0	2	1	2	0	3
$\text{Oracle}_{\text{bitwiseOR},5}(k)$	1	0	0	1	0	1	0	1	1	1	0	1

Table 2: Digits for $U = 5$. $\text{Oracle}_{\text{count},5}(12) = 3$ from $n \in \{2, 3, 4\}$; $\text{Oracle}_{\text{count},5}(4) = 1$ since $n = 4$ itself is excluded. Without the marker, $k = 1$ has digit 0 and looks prime.

Proof. $m = 1$ has marked digit 1. A prime has no divisor in $[2, m-1]$. A composite $m < (U+1)^2$ has a prime factor $q \le \sqrt{m} < U + 1$, so $q \le U$, $q < m$, and $\text{Oracle}_{\text{count},U}(m) \ge 1$. ■

Thus $U = \lceil \sqrt{N} \rceil$ suffices for every $m < N$. Position 0 is not a digit of a fractional expansion and is excluded by one comparator.

Proposition 4 (Digit extraction) *For $b > W_U$ and $k \ge 1$: $\text{Oracle}_{w,U}(k) = \lfloor b\{b^{k-1}\Sigma^w_{U,b}\}\rfloor$, where $\{\cdot\}$ is the fractional part.*

Proof. Multiplying by b^{k-1} moves digits $1, \dots, k-1$ to the integer part, so $\{b^{k-1}\Sigma\} = \sum_{j\ge k} \text{Oracle}_{w,U}(j) b^{-(j-k+1)}$, a valid expansion with first digit $\text{Oracle}_{w,U}(k)$ and tail < 1. ■

The exponent is $k - 1$, not k.

2.1 The bitwise-OR encoding

Counting digits needs "headroom": $b > U - 1$. If one only needs to know whether a position has been struck, the sum can be replaced by a digit-wise OR, which needs no such headroom and works in base 2.

Definition 5 *Let $b = 2^r$, $r \ge 1$. Each $S_n(b)$ has digits in $\{0, 1\}$, each occupying the lowest bit of a block of r bits, so the bitwise OR of binary expansions equals the digit-wise OR. Define*

$$\Theta_{U,b} := b^{-1} \vee S_2(b) \vee S_3(b) \vee \cdots \vee S_U(b) \qquad (\vee = \textit{bitwise OR}). \tag{3}$$

Theorem 6 (OR sieve) *For every $b = 2^r$ ($r \ge 1$), $\Theta_{U,b} = \sum_{k\ge1} \text{Oracle}_{\text{bitwiseOR},U}(k)\, b^{-k}$ with*

$$\text{Oracle}_{\text{bitwiseOR},U}(k) = [\,k = 1\,] \;\vee\; [\,\exists\, n \in [2, U] :\; n \mid k,\; n < k\,] \in \{0, 1\}. \tag{5}$$

Hence for $1 \le m < (U+1)^2$, $\text{Oracle}_{\text{bitwiseOR},U}(m) = 0$ iff m is prime; and $\text{Oracle}_{\text{bitwiseOR},U}(k) = \min\{1, \text{Oracle}_{\text{markedCount},U}(k)\}$. The extraction identity $\text{Oracle}_{\text{bitwiseOR},U}(k) = \lfloor b\{b^{k-1}\Theta_{U,b}\}\rfloor$ holds.

Proof. OR acts independently in each digit position and each S_n has a 1 exactly at the positions jn, $j \ge 2$. The digits are not eventually $b - 1$ (they vanish at primes $> U$), so the extraction argument of Proposition 4 applies. ■

The OR oracle is the same circuit as the counting oracle with a one-bit accumulator in place of a counter (Lemma 7); it is defined and carry-free in bases 2, 4, 16, etc., with no condition tying b to U. It does not give divisor counts.

3 Quantum oracles and prime counting

Fix $N = 2^\alpha$, $U = \lceil\sqrt{N}\rceil$, and let $b = 2$ (OR encoding) or $b = U$ (counting encoding). Define $\mathrm{Oracle}_{\mathrm{prime},U}(m) = 1$ iff $m \geq 1$ and $\mathrm{Oracle}_{\mathrm{bitwiseOR},U}(m) = 0$ (equivalently $\mathrm{Oracle}_{\mathrm{markedCount},U}(m) = 0$). By Corollary 3 and Theorem 6, $\mathrm{Oracle}_{\mathrm{prime},U}(m) = 1$ iff m is prime, for $0 \leq m < N$. The unitary is $\mathbf{O}_{\mathrm{prime}}|m\rangle|y\rangle = |m\rangle|y \oplus \mathrm{Oracle}_{\mathrm{prime},U}(m)\rangle$.

Lemma 7 (Reversible implementation) $\mathbf{O}_{\mathrm{prime}}$ *can be implemented with* $O(U\alpha^2)$ *gates and* $O(\alpha)$ *ancilla qubits. The divisor-count oracle* $|m\rangle|0\rangle \mapsto |m\rangle|\mathrm{Oracle}_{\mathrm{count},U}(m)\rangle$ *needs additionally* $\lceil \log_2 U \rceil$ *qubits.*

Proof. By Theorems 2 and 6 the digit equals $\bigvee_n [n \mid m,\ n < m]$ (or the sum). Each term needs $m \bmod n$, an $O(\alpha^2)$-gate reversible computation. Loop over $n = 2, \ldots, U$, compute the flag, OR it into one accumulator bit (or add it to a counter), uncompute the remainder, copy out and uncompute (Bennett). The b-adic number of length $\Theta(m)$ digits is never stored. ■

Remark 8 (Powers of the base and binary exponentiation) *Written termwise, the extraction of Proposition 4 needs* b^{k-1} *with* k *in superposition:* $\{b^{k-1}S_n(b)\} = \big(b^{k-1-n} \bmod (b^n - 1)\big)/(b^n - 1)$. *A power of the base with an exponent held in a register is computed by fast binary (square-and-multiply) exponentiation as in Shor's algorithm: precompute* $b^{2^i} \bmod (b^n - 1)$ *classically and multiply in the factors controlled on the exponent bits,* $O(\alpha)$ *modular multiplications, polynomial in the bit-length of the modulus. The same holds for a shift by* b^c *when* c *is held in a register (Appendix E); for a constant shift* c *one simply adds* c *to* k. *The bit-length of the modulus is* $n \log_2 b$, *which is exponential in* α *for* n *up to* $U \approx \sqrt{N}$, *so this route is not polynomial overall. The simplification* $b^{k-1-n} \bmod (b^n - 1) = b^{(k-1-n) \bmod n}$ *reduces each term to computing* $k \bmod n$, *which is what Lemma 7 uses.*

Theorem 9 (Quantum counting of primes) *Let* $N \geq 32$ *and* $0 < \varepsilon \leq 1$. *Quantum counting [4, 5] with* $P = \lceil 4\pi\sqrt{2 \ln N}/\varepsilon \rceil$ *applications of* $\mathbf{O}_{\mathrm{prime}}$ *outputs* $\tilde{t}$ *with* $|\tilde{t} - \pi(N-1)| \leq \varepsilon\, \pi(N-1)$ *with probability* $\geq 8/\pi^2$ *(amplified to* $1 - \delta$ *by a median of* $O(\log 1/\delta)$ *runs), using* $O(\sqrt{\ln N}\, \varepsilon^{-1} \sqrt{N} \alpha^2)$ *gates.*

Proof. With $t = \pi(N-1)$ marked items among N, amplitude estimation gives $|\tilde{t} - t| \leq 2\pi\sqrt{t(N-t)}/P + \pi^2 N/P^2$ with probability $\geq 8/\pi^2$ [5]. For $N \geq 32$, $t \geq N/(2 \ln N)$ (Rosser–Schoenfeld). Then $2\pi\sqrt{tN}/P \leq \varepsilon t/2$ once $P \geq 4\pi\sqrt{N/t}/\varepsilon$, which holds since $\sqrt{N/t} \leq \sqrt{2 \ln N}$; and $\pi^2 N/P^2 \leq \varepsilon t/2$ once $P \geq 2\pi\sqrt{\ln N/\varepsilon}$, which is implied. Multiply by the cost in Lemma 7. ■

Algorithm 1 (quantum counting of primes below N; the algorithm behind Theorem 9). Let $|\psi\rangle = N^{-1/2} \sum_{x=0}^{N-1} |x\rangle$, let $\mathbf{S}_{\mathrm{prime}}|x\rangle = (-1)^{\mathrm{Oracle}_{\mathrm{prime},U}(x)}|x\rangle$ be the phase version of $\mathbf{O}_{\mathrm{prime}}$ (apply $\mathbf{O}_{\mathrm{prime}}$ with the target qubit in $|-\rangle$), and let $G = (2|\psi\rangle\langle\psi| - I)\, \mathbf{S}_{\mathrm{prime}}$ be the Grover operator. With $t = \pi(N-1)$ and $\sin^2\varphi = t/N$, write $|\psi\rangle = \sin\varphi|\mathrm{good}\rangle + \cos\varphi|\mathrm{bad}\rangle$, where $|\mathrm{good}\rangle$ and $|\mathrm{bad}\rangle$ are the normalised superpositions of the primes and of the non-primes below N. Then $G^j|\psi\rangle = \sin((2j+1)\varphi)|\mathrm{good}\rangle + \cos((2j+1)\varphi)|\mathrm{bad}\rangle$, and G has eigenvalues $e^{\pm 2i\varphi}$. Let $P = \lceil 4\pi\sqrt{2 \ln N}/\varepsilon \rceil$ and let $P' = 2^m \geq P$ be the next power of two ($m = \lceil \log_2 P \rceil$).

1. **Initialisation and uniform superposition.** Take a counting register of m qubits and a data register of α qubits, both in $|0\rangle$, and apply Hadamard gates to both:

$$|\psi_1\rangle = \frac{1}{\sqrt{P'}} \sum_{j=0}^{P'-1} |j\rangle|\psi\rangle = \frac{1}{\sqrt{P'N}} \sum_{j=0}^{P'-1} \sum_{x=0}^{N-1} |j\rangle|x\rangle. \tag{4}$$

2. **Controlled Grover iterations (prime oracle).** Apply G^{2^i} to the data register controlled on bit i of the counting register, for $i = 0, \ldots, m-1$; in total $P'-1$ applications of G, each containing one application of $\mathbf{O}_{\text{prime}}$:

$$|\psi_2\rangle = \frac{1}{\sqrt{P'}} \sum_{j=0}^{P'-1} |j\rangle\, G^j|\psi\rangle = \frac{1}{\sqrt{P'}} \sum_{j=0}^{P'-1} |j\rangle\Big(\sin\big((2j+1)\varphi\big)|\text{good}\rangle + \cos\big((2j+1)\varphi\big)|\text{bad}\rangle\Big). \tag{6}$$

   Equivalently, $|\psi\rangle$ is an equal-weight superposition of the eigenvectors $|\omega_\pm\rangle$ of G, so $|\psi_2\rangle = \frac{1}{\sqrt{2P'}} \sum_j |j\rangle\,(c_+ e^{2ij\varphi}|\omega_+\rangle + c_- e^{-2ij\varphi}|\omega_-\rangle)$ with $|c_\pm| = 1$: the phase 2φ is written into the counting register.

3. **Inverse quantum Fourier transform.** Apply $\text{QFT}^\dagger_{P'}$ to the counting register, where

$$\text{QFT}^\dagger_{P'}|j\rangle = \frac{1}{\sqrt{P'}} \sum_{k=0}^{P'-1} e^{-2\pi ijk/P'}|k\rangle. \tag{7}$$

   The two terms of $|\psi_2\rangle$ are then concentrated near $k \approx P'\varphi/\pi$ and $k \approx P' - P'\varphi/\pi$. Measure the counting register to obtain y.

4. **Classical post-processing.** Output $\tilde{t} = N\sin^2(\pi y/P')$ (the same value for y and $P'-y$). By the analysis of amplitude estimation [5], with probability at least $8/\pi^2$

$$|\tilde{t} - t| \le \frac{2\pi\sqrt{t(N-t)}}{P'} + \frac{\pi^2 N}{(P')^2}. \tag{8}$$

   For $N \ge 32$, $t \ge N/(2\ln N)$ (Rosser–Schoenfeld), and by the choice of P (proof of Theorem 9) the right-hand side is at most εt. (This step is an analysis of the output, not a further quantum operation.)

5. **Amplification.** Repeat Steps 1–4 independently $K = O(\log 1/\delta)$ times and output the median of the estimates $\tilde{t}$, which satisfies $|\tilde{t} - \pi(N-1)| \le \varepsilon\,\pi(N-1)$ with probability at least $1-\delta$.

Cost. One run applies G at most $P'-1 < 2P$ times; each application costs one $\mathbf{O}_{\text{prime}}$ ($O(U\alpha^2) = O(\sqrt{N}\alpha^2)$ gates by Lemma 7) plus the reflection about $|\psi\rangle$ ($O(\alpha)$ gates), and the inverse QFT costs $O(m^2)$ gates. The total is $O(\sqrt{\ln N}\,\varepsilon^{-1}\sqrt{N}\alpha^2)$ gates per run, multiplied by K, on $m + O(\alpha)$ qubits.

Remark 10 (On efficiency of the sieve oracle) *Relative error ε means additive error $\varepsilon\pi(N) \approx \varepsilon N/\ln N$, so comparing with* $\text{li}(N)$ *(error $O(\sqrt{N}\ln N)$ under the Riemann Hypothesis) needs $\varepsilon \ll \ln N/\sqrt{N}$ lowever this comparison is not very direct in the sense that our algorithm*

does not assume the truth of the Riemann Hypothesis. The oracle of Lemma 7 costs $\tilde{\Theta}(\sqrt{N})$ *gates because it is a version trial division. A deterministic polynomial-time primality test (AKS [1]) gives a* $\mathrm{poly}(\alpha)$*-gate oracle, so the same counting procedure runs in* $\mathrm{poly}(\alpha)\sqrt{\ln N}/\varepsilon$*. The digit encoding is therefore a* representation *of sieve data, not an efficiency gain.*

Remark 11 (Per-term cost, space and time) *The oracle is polynomial* per term*; the number of terms* $U \approx \sqrt{N}$ *is the exponential factor. It uses* $O(\alpha)$ *qubits, compared with* $O(N)$ *(or* $O(\sqrt{N})$ *segmented) bits for the classical sieve, but this saving comes from computing digits on demand, not from superposition: a classical trial-division loop also uses* $O(\log N)$ *space. In time, approximate counting at relative error* ε *costs* $\tilde{O}(\sqrt{N}/\varepsilon)$ *gates, below the* $\tilde{O}(N)$ *of the exact classical sieve, but exact counting requires* $\Omega(N/\sqrt{\ln N})$ *queries (Theorem 23), and classical Monte Carlo sampling with a polynomial-time primality test estimates* $\pi(N)$ *in* $\mathrm{poly}(\alpha)\ln N/\varepsilon^2$ *time. With the same oracle, quantum counting improves the number of samples quadratically (*$\sqrt{\ln N}/\varepsilon$ *versus* $\ln N/\varepsilon^2$*) and nothing more.*

4 Divisor sums and odd perfect numbers

Take $w_n = n$. Then $W_U = U(U+1)/2 - 1$, so $b \geq U(U+1)/2$ is carry-free, and the digit is $\mathrm{Oracle}_{\mathrm{divisorSum},U}(k) = \sum_{n|k,\, 2\leq n\leq U,\, n<k} n$.

Proposition 12 *(a) If* $U \geq \lfloor k/2 \rfloor$ *and* $k \geq 2$ *then* $\mathrm{Oracle}_{\mathrm{divisorSum},U}(k) = \sigma(k) - k - 1$*. (b) If* k *is odd and* $U \geq \lfloor k/3 \rfloor$ *then also* $\mathrm{Oracle}_{\mathrm{divisorSum},U}(k) = \sigma(k) - k - 1$*. Consequently, for odd* $k \geq 3$ *with* $U \geq \lfloor k/3 \rfloor$*:* k *is perfect iff* $\mathrm{Oracle}_{\mathrm{divisorSum},U}(k) = k - 1$*, and abundant iff* $\mathrm{Oracle}_{\mathrm{divisorSum},U}(k) \geq k$*.*

Proof. (a) Every divisor of k other than $1, k$ lies in $[2, k/2]$. (b) For odd k and a divisor $d < k$, the cofactor $k/d > 1$ is odd, hence ≥ 3, so $d \leq k/3$. Then $\sigma(k) = 2k \iff \mathrm{Oracle}_{\mathrm{divisorSum},U}(k) = k - 1$. (The value is $\sigma(k)$ minus 1 *and* minus k.) ■

Proposition 13 (Paired divisor sum) *For* $k \geq 2$ *define*

$$\mathrm{Oracle}_{\mathrm{pairedDivisorSum}}(k) = \sum_{\substack{d|k \\ 2\leq d,\ d^2<k}} \left(d + \frac{k}{d}\right) + \left[\, k = m^2,\ m \geq 2 \,\right] m.$$

Then $\mathrm{Oracle}_{\mathrm{pairedDivisorSum}}(k) = \sigma(k) - k - 1$*, and the sum has at most* $\lfloor \sqrt{k} \rfloor - 1$ *candidate values of* d*.*

Proof. The divisors of k other than 1 and k are the integers $e \mid k$ with $2 \leq e \leq k/2$. Pair each such e with k/e: if $e < \sqrt{k}$ the partner $k/e > \sqrt{k}$ is also such a divisor (it is ≥ 2 and $\leq k/2$ because $e \geq 2$), and if $e = \sqrt{k}$ the pair is a single divisor. Listing each pair once by its smaller member d ($2 \leq d$, $d^2 < k$), and adding the square root once when k is a square, gives the sum of all divisors of k other than 1 and k. ■

This is a circuit-level oracle (the bound $d^2 < k$ and the cofactor k/d depend on the position k), not a fixed-U digit of a single number. It needs only $d \leq \lfloor \sqrt{N-1} \rfloor$ for $k < N$, so its cost is $O(\sqrt{N}\,\alpha^2)$ gates per evaluation.

4.1 An odd perfect number oracle and existence search

An odd perfect number is an odd k with $\sigma(k) = 2k$; none is known. Fix $N = 2^\alpha$ and $U = \lfloor (N-1)/3 \rfloor$. Define

$$\mathrm{Oracle}_{\mathrm{oddPerfect},U}(k) = 1 \iff k \text{ odd},\ k \geq 3,\ \mathrm{Oracle}_{\mathrm{divisorSum},U}(k) = k - 1,$$

which by Proposition 12(b) marks exactly the odd perfect numbers in $[3, N)$ (*Variant A*, the digit oracle). *Variant B* replaces $\mathrm{Oracle}_{\mathrm{divisorSum},U}(k)$ by $\mathrm{Oracle}_{\mathrm{pairedDivisorSum}}(k)$ of Proposition 13, with the same predicate $\sigma(k) = 2k$. Let M be the number of odd perfect numbers in $[3, N)$. Since we only want to know *whether* one exists, and not how many, we use the *existence* version of amplitude amplification, which needs no phase estimation, no counting register and no QFT. The following are the first quantum algorithms (to the best of our knowledge) for finding the existence of an odd perfect number using quantum computers; maybe in future their efficiency,regarding space complexity or time complexity, can be improved. Note there is more than one choice of oracle hence giving more than one algorithm for odd perfect numbers.

Algorithm 2 (quantum existence search for odd perfect numbers below N).

1. **Uniform superposition.** Prepare $|\psi\rangle = N^{-1/2} \sum_{k=0}^{N-1} |k\rangle$ on α qubits (to restrict to odd k, write $k = 2x + 1$ on $\alpha - 1$ qubits).

2. **Odd-perfect oracle.** Apply the phase oracle $\mathbf{O}_{\mathrm{oddPerfect}}|k\rangle = (-1)^{\mathrm{Oracle}_{\mathrm{oddPerfect},U}(k)}|k\rangle$: with a 2α-bit register s initialised to 0, for $n = 2, \ldots, U$ compute $r = k \bmod n$ and, if $r = 0$ and $n < k$, add n to s (so $s = \mathrm{Oracle}_{\mathrm{divisorSum},U}(k)$ at the end; $s \leq \sigma(k) < 2k(1 + \ln k)$ fits in 2α bits); then compute the bit $[\, s = k - 1\,] \wedge [k \text{ odd}] \wedge [k \geq 3]$, apply a phase flip, and uncompute everything. (Variant B: loop instead over $d = 2, \ldots, \lfloor \sqrt{N-1} \rfloor$; if $d \mid k$ and $d^2 < k$ add $d + k/d$ to s, and if $d^2 = k$ add d; the loop count and the comparator $d^2 \leq k$ replace the bound $U \approx N/3$.)

3. **Existence search (BBHT schedule).** With the Grover operator $G = (2|\psi\rangle\langle\psi| - I)\,\mathbf{O}_{\mathrm{oddPerfect}}$ and $\lambda = 6/5$, for stage $j = 0, 1, 2, \ldots$ draw r uniformly from $\{0, \ldots, \lceil \lambda^j \rceil - 1\}$, apply G^r to $|\psi\rangle$, measure k, and *verify classically* that k is odd and $\sigma(k) = 2k$. If verified, output k and stop. Stop with "none found" when the total number of applications of G exceeds $c\sqrt{N}$ for a suitable constant c [3].

Theorem 14 *Algorithm 2 has no false positives (every output is classically verified). If $M \geq 1$ it outputs a perfect odd $k < N$ with probability $\geq 2/3$ using expected $O(\sqrt{N/M})$ applications of $\mathbf{O}_{\mathrm{oddPerfect}}$; in all cases it uses $O(\sqrt{N})$ applications, each using $O(\alpha)$ qubits. A Variant A application costs $O(N\alpha^2)$ gates, so $\tilde{O}(N^{3/2})$ gates in total; a Variant B application costs $O(\sqrt{N}\,\alpha^2)$ gates, so $O(N\alpha^2) = \tilde{O}(N)$ gates in total. Repetition reduces the one-sided error.*

Proof. Standard analysis of amplitude amplification for an unknown number of solutions [3]; the per-query cost is Lemma 7 with $U \approx N/3$ for Variant A, and $\lfloor \sqrt{N-1} \rfloor$ iterations of $O(\alpha^2)$ gates for Variant B (Proposition 13). The total is the product $O(\sqrt{N}) \times$ (per-query cost). ∎

Remark 15 (Cost, and what is and is not new) *(i) With the fixed-U digit oracle (Variant A) a single digit has $U \approx N/3$ terms, because a fixed bound $U \approx \sqrt{N}$ captures only the divisors up to $\sqrt{N}$ and not $\sigma(k) - k - 1$; the total is $\tilde{O}(N^{3/2})$, worse than the classical $O(N \log N)$ divisor-sum sieve. Variant B, which pairs each divisor $d \leq \sqrt{k}$ with its cofactor, needs only $\sqrt{N}$ iterations per*

query and gives $O(\sqrt{N}) \times O(\sqrt{N}\alpha^2) = \tilde{O}(N)$ *in total: the same footing as classical exhaustive verification, but not better. (Variant B is not a fixed-weight digit: the cofactor* k/d *and the bound* $d^2 < k$ *depend on the position* k*; the higher-order encodings of Appendix C give cofactor weights but over all* $d \le U$ *at once.) Replacing the oracle by a potential circuit that finds* $\sigma(k)$ *from the factorisation of* k *(an Shor's type algorithm/circuit with error reduction and uncomputation,* $\mathrm{poly}(\alpha)$ *gates) gives* $\tilde{O}(\sqrt{N}\,\mathrm{poly}(\alpha))$*, a quadratic saving over classical enumeration of all odd* $k < N$*; this is a feature of the search structure, not of the digit encoding. (ii) No odd perfect number lies below* 10^{1500} *[16], so Algorithm 2 with* $N < 10^{1500}$ *returns "none found"; a nontrivial run uses an interval* $[H, H+2^\alpha)$ *with* $H > 10^{1500}$*, where* $U \approx H/3$ *makes the digit oracle impractical and only the potential factoring-based oracle applies. Structural theorems (Euler form) can shrink the domain further. (iii) The search is generic amplitude amplification over a predicate checkable in polynomial time. We are not aware of a dedicated published quantum algorithm for odd perfect numbers, but we claim only the explicit digit-encoded oracle* $\mathrm{Oracle}_{\mathrm{oddPerfect},U}$ *and its analysis. (iv) "None found" is probabilistic and covers only* $[3, N)$*; it never proves non-existence. (v)* Provable bounds. *The costs in Theorem 14 and in (i) are unconditional worst-case bounds. Classical plain enumeration also has a provable bound,* $O(N \log N)$ *with a divisor-sum sieve, which the factoring-oracle variant improves quadratically (*$\tilde{O}(\sqrt{N}\,\mathrm{poly}(\alpha))$*) while the digit-oracle variants do not. The classical searches that reach the largest known bounds [16] exploit the structure of odd perfect numbers (branching on prime factorisations) and, to our knowledge, have no proven running-time bound; in practice they are far faster than Algorithm 2 as stated. So the factoring-oracle variant is the best provable bound we know for this exhaustive-range formulation, not the fastest method in practice.*

5 Periodicity of the digit sequences

For $k > U$ the condition $n < k$ is automatic, so $\mathrm{Oracle}_{w,U}(k) = \sum_{n \le U} w_n [n \mid k]$ and $\mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k) = [\exists\, n \le U : n \mid k]$.

Proposition 16 *(a) For positive weights,* $(\mathrm{Oracle}_{w,U}(k))_{k>U}$ *is purely periodic with minimal period* $L_U = \mathrm{lcm}(2, \ldots, U)$*. (b)* $(\mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k))_{k>U}$ *is purely periodic with minimal period the primorial* $U\# = \prod_{p \le U} p$*.*

Proof. (a) L_U is a period. If T is a period: the maximum W_U is attained iff every $n \le U$ divides k, i.e. $L_U \mid k$; taking $k = 2L_U > U$, $\mathrm{Oracle}_{w,U}(k+T) = W_U$ forces $L_U \mid T$.

(b) The digit is 0 iff $\gcd(k, U\#) = 1$, so $U\#$ is a period. Let T be a period and suppose a prime $p \le U$ does not divide T. Choose $k > U$ with $k \equiv -T \pmod p$ (nonzero mod p) and $k \equiv 1 \pmod q$ for every other prime $q \le U$. Then $\gcd(k, U\#) = 1$ and the digit at k is 0, but $p \mid k+T$, so the digit at $k+T$ is 1, a contradiction. Hence $U\# \mid T$. ∎

Thus the primorial appears only for the OR (flag) sequence; the counting sequences have the larger period L_U (e.g. $U = 4$: 12 versus 6). The following is thus an algorithm that builds a bridge between the quantum Fourier Transform and the prime counting function.

5.1 A quantum period-finding algorithm

Let f be a function on the integers $x \ge 0$ with values in a finite set (encoded in bits), purely periodic with exact period r and *injective on one period*. For the digit sequences we take $f(x) = h(x + U + 1)$, where h is one of the maps below; the shift is needed because the digit sequences are purely periodic only for $k > U$. Candidates for h:

1. the shifted stacking map $\mathrm{Oracle}_{\mathrm{stacked},U}$ of Section 6 (period $r = L_U$; injective, Theorem 20);

2. the OR window map $\mathrm{Oracle}_{\mathrm{windowOR},U,S}(k) = \big(\mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k), \mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k+1), \ldots, \mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k+S-1)\big)$ (period $r = U\#$; injective on a period only if S is large enough, which we have verified numerically for $U \le 9$ but not proved);

3. a *single* digit oracle is not admissible: by Theorem 19 it is not injective on a period, and the algorithm may then return only a proper divisor of the period.

Since $L_U \le 3^U$ and $U\# < 4^U$, we have $r < 4^U$. Put $n = 4U + 1$ and $Q = 2^n \ge 2r^2$.

Algorithm 3 (period finding for a digit sequence).

1. **Initialisation and uniform superposition.** Take a first register of n qubits (indexing $x \in \{0, \ldots, Q-1\}$) and a second register for the value of f. Apply Hadamard gates to the first register:
$$|\psi_1\rangle = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle|0\rangle.$$

2. **Sequence oracle.** Apply the reversible circuit $\mathbf{O}_f : |x\rangle|0\rangle \mapsto |x\rangle|f(x)\rangle$ (Lemma 7 for each digit, repeated for each shift):
$$|\psi_2\rangle = \frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle|f(x)\rangle.$$

3. **Quantum Fourier transform.** Apply $\mathrm{QFT}_Q : |x\rangle \mapsto Q^{-1/2} \sum_{y=0}^{Q-1} e^{2\pi i x y/Q} |y\rangle$ to the first register:
$$|\psi_3\rangle = \frac{1}{Q} \sum_{y=0}^{Q-1} \sum_{x=0}^{Q-1} e^{2\pi i x y/Q} |y\rangle|f(x)\rangle.$$
Writing $x = x_0 + wr$ with $0 \le x_0 < r$, the amplitude of $|y\rangle|f(x_0)\rangle$ is $\frac{1}{Q} e^{2\pi i x_0 y/Q} \sum_w E^w$ with $E = e^{2\pi i r y/Q}$, a geometric sum that is large only when ry/Q is close to an integer. Measuring the first register gives y with y/Q close to j/r for some $j \in \{0, \ldots, r-1\}$; for each j the integer nearest to jQ/r is observed with probability at least $4/(\pi^2 r)$. (Measuring the second register first is optional and does not change the distribution of y.)

4. **Continued fractions.** Since $|y/Q - j/r| \le 1/(2Q) \le 1/(4r^2)$, the fraction j/r is a convergent of y/Q. Expand y/Q in a continued fraction and take the denominator r' of the last convergent with denominator $< \sqrt{Q/2}$; then $r' = r/\gcd(j, r)$ divides r.

5. **Validation and repetition.** Keep ρ equal to the lcm of all denominators r' obtained so far. After each run test, with one classical evaluation, whether $f(x_0 + \rho) = f(x_0)$ for some fixed x_0. Because f is injective on a period, this test holds iff $r \mid \rho$, and since $\rho \mid r$ it certifies $\rho = r$. If the test fails, repeat Steps 1–4.

Theorem 17 *Let f be purely periodic with exact period r and injective on a period, with $r^2 \le Q/2$. Each run of Algorithm 3 yields $r' = r$ with probability $\Omega(1/\log\log r)$, so $O(\log\log r)$ runs return r with probability $\ge 2/3$ (fewer with the lcm rule of Step 5, which succeeds whenever the*

observed j values have gcds that jointly cancel). Each run uses $n = O(U)$ qubits and $O(n^2)$ gates for the QFT plus the cost of $\mathbf{O}_f$. For $h = \mathrm{Oracle}_{\mathrm{stacked},U}$ this is $O(U^4)$ gates per run (via standard arithmetic), and the result is unconditional. For $h = \mathrm{Oracle}_{\mathrm{windowOR},U,S}$ the same holds provided the window map is injective on a period; otherwise the output divides $U\#$ and Step 5 cannot certify it.

Proof. For $\gcd(j, r) = 1$, the probability computed in Step 3 and the convergent argument in Step 4 give $r' = r$. A uniform j is coprime to r with probability $\varphi(r)/r = \Omega(1/\log\log r)$. The certification in Step 5 is the injectivity argument above. The gate counts are Lemma 7 with n-bit inputs: U^2 divisibility tests of $O(n^2)$ gates for the stacked map. ■

6 Transferring the Zig–Zag construction

In the Pisano setting [17] the state $v_n = A^n v_0 \bmod m$ of an invertible linear recurrence is a faithful coordinate on the cycle, and stacking shifted copies in disjoint digit blocks yields an encoding conjectured to be injective on one period. Sieve digits are different in kind.

Lemma 18 *For $k > U$ and $n \le U$: $[n \mid k] = [n \mid \gcd(k, L_U)]$. Hence every weighted digit depends only on $\gcd(k, L_U)$, and the OR digit only on whether $\gcd(k, U\#) = 1$.*

Theorem 19 (No digit-oracle Zig–Zag) *Let $U \ge 3$ and let F be any function of finitely many weighted digit oracles and the OR digit,*

$$F(k) = \Phi\big(\mathrm{Oracle}_{w^{(1)},U}(k), \ldots, \mathrm{Oracle}_{w^{(J)},U}(k), \mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k)\big),$$

with any weights, any J and any Φ. Then F takes at most $\tau(L_U) < L_U$ values and $F(k) = F(k')$ whenever $k \equiv -k' \pmod{L_U}$. In particular F is not injective on any window of L_U consecutive integers $> U$.

Proof. By Lemma 18, $F(k)$ is a function of $\gcd(k, L_U)$, and $\gcd(k, L_U) = \gcd(-k, L_U)$. For $U \ge 3$, $L_U \ge 6$, so the residues 1 and $L_U - 1$ are distinct, and a window of L_U consecutive integers contains one integer from each. ■

So a direct transfer of the injectivity conjecture to digit oracles is *false* for every $U \ge 3$. Numerically, for $U = 3, \ldots, 12$ the weighted oracle $\sum_n 2^n[n \mid k]$ takes exactly $\tau(L_U)$ values on a period $(4, 6, 12, 12, 24, 32, 48, 48, 96, 96)$ against periods $6, 12, 60, 60, 420, 840, 2520, 2520, 27720, 27720$. Adding digit blocks with overlap, e.g. $\mathrm{Oracle}_1(k) + b\,\mathrm{Oracle}_2(k+1)$, adds shifted digits (collisions and carries) and cannot create information that is not present.

How to extend for injective Zig-Zig Stacking: shifted stacking in disjoint blocks. Information about k itself, not only $\gcd(k, L_U)$, enters through *shifts*. Let $\mathrm{Oracle}_{\mathrm{subsetCode},U}(k) = \sum_{n=2}^{U} 2^n[n \mid k] < B := 2^{U+1}$ (weights $w_n = 2^n$, base $b \ge B$) and

$$\mathrm{Oracle}_{\mathrm{stacked},U}(k) = \sum_{s=0}^{U-1} B^{\,s}\,\mathrm{Oracle}_{\mathrm{subsetCode},U}(k+s).$$

Theorem 20 (Injective Zig–Zag stacking) *For $k > U$: $\mathrm{Oracle}_{\mathrm{stacked},U}(k + L_U) = \mathrm{Oracle}_{\mathrm{stacked},U}(k)$, and $\mathrm{Oracle}_{\mathrm{stacked},U}(k) = \mathrm{Oracle}_{\mathrm{stacked},U}(k')$ iff $k \equiv k' \pmod{L_U}$. So the stacked map is injective on one period and has minimal period exactly L_U.*

Proof. Periodicity is Proposition 16(a). If the stacked values agree, then, as the blocks are base-B digits with $B > \max \mathrm{Oracle}_{\mathrm{subsetCode},U}$, $\mathrm{Oracle}_{\mathrm{subsetCode},U}(k+s) = \mathrm{Oracle}_{\mathrm{subsetCode},U}(k'+s)$ for $0 \le s \le U-1$, i.e. $[n \mid k+s] = [n \mid k'+s]$ for all $n \le U$. For each $n \le U$ there is a unique $s_n \in [0, n-1] \subseteq [0, U-1]$ with $n \mid k+s_n$, so also $n \mid k'+s_n$, and $k \equiv k' \equiv -s_n \pmod{n}$. This holds for every $n \le U$, so $L_U \mid k-k'$. ■

Numerically the stacked map takes exactly L_U values on a period for $U = 2, \ldots, 12$. The shift s plays the role of the shift J in the Pisano construction; the sieve setting shows that without shifts no injectivity is possible.

Remark 21 (OR version) *The OR sequence takes only two values, so a single digit is far from injective on its period $U\#$. Stacking shifted windows $(\mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k+s))_{s<S}$ is the natural analogue; numerically the window map is injective on a period for $U = 2, \ldots, 9$ with $S = 1, 4, 4, 14, 14, 38, 38, 38$ respectively. We have not proved this, nor how S grows with U. Since the period has $\approx U/\ln 2$ bits, $S \gtrsim U/\ln 2$ is necessary.*

Corollary 22 (Period finding) *Let $t = \lceil 2\log_2 L_U \rceil + 1 = \Theta(U)$ and $f(x) = \mathrm{Oracle}_{\mathrm{stacked},U}(x + U + 1)$. The standard period-finding circuit on f outputs L_U with probability $\Omega(1/\log\log L_U)$ per run, using $\mathrm{poly}(U)$ gates ($O(U^4)$ with standard arithmetic), unconditionally.*

Proof. f is purely periodic of exact period L_U and injective on it (Theorem 20), which is Shor's setting [20]: the measured value is within 2^{-t-1} of j/L_U with j uniform, and the reduced fraction has denominator L_U when $\gcd(j, L_U) = 1$, which has probability $\varphi(L_U)/L_U = \Omega(1/\log\log L_U)$. Evaluating f takes U^2 divisibility tests on $\Theta(U)$-bit integers. ■

This is Algorithm 3 (Section 5.1) with $h = \mathrm{Oracle}_{\mathrm{stacked},U}$.

7 What period recovery gives, and what it does not

The primes dividing L_U are exactly the primes $\le U$, so $\pi(U) = \omega(L_U)$. Corollary 22 followed by factoring L_U (a $\Theta(U)$-bit number) computes $\pi(U)$ in time $\mathrm{poly}(U)$, unconditionally. But:

1. L_U is a fixed function of U; the algorithm learns nothing not determined by the input U.
2. Counting primes up to N needs $U = N$, so the running time is $\mathrm{poly}(N)$, exponential in the input length α, and no better than the classical $\tilde{O}(N)$ sieve.
3. The construction does not need the primes in advance: the stacked sequence is defined from divisibility by *all* $n \le U$, and the period L_U is discovered from the sequence, after which factoring L_U reveals the primes. The algorithm is therefore not circular. It is, however, *a different new quantum intrinisic route compared to the classical algorithm for the same task*: the same output is obtained classically in $\mathrm{poly}(U)$ time, since L_U is computed from U by the iterated lcm $\mathrm{lcm}(\mathrm{lcm}(2,3),4), \ldots$ (gcd computations, no primes needed) and its prime factors are then found by trial division by $2, \ldots, U$.

Consequently the period construction is a also another worked example of how Zig–Zag stacking restores injectivity.

8 Inverse prime counting

Given a prime $p = P_L$, determine $L = \pi(p)$. With the oracle of Section 3 on $N = p + 1$ items, L is the number t of marked items.

Theorem 23 (Oracle lower bound) *Any quantum algorithm accessing the primality predicate only as a black box and outputting the number t of marked items among N exactly needs $\Omega(\sqrt{t(N-t)})$ queries; more generally, approximating t to additive error $\Delta \geq 1$ needs $\Omega(\sqrt{N/\Delta} + \sqrt{t(N-t)}/\Delta)$ [15]. The amplitude-estimation bound used in Theorem 9 is tight in this sense [5, 15].*

For $t = \pi(p) \sim p/\ln p$ this is $\Omega(p/\sqrt{\ln p})$ queries, essentially linear in p. A counting-based determination of L is therefore not an $O(\sqrt{p})$ algorithm; with the sieve oracle it costs $\tilde{\Omega}(p^{3/2})$ gates, against $\tilde{O}(p^{2/3})$ [10, 6] and $O(p^{1/2+\epsilon})$ [11] classically. Grover search [8] locates a marked element but does not count them.

Proposition 24 (Interval version) *If P_L lies in $[H, H + 2^\alpha)$, counting primes in the interval yields $\pi(P_L) - \pi(H)$, not L; computing L additionally requires $\pi(H)$, itself a prime-counting problem of size $H \geq 2^\alpha$.*

9 Conclusion

The digit encoding of the sieve gives exact oracles for primality (counting and bitwise-OR forms, with a marker for 1), divisor counts and divisor sums. It supports quantum counting and an explicit quantum existence search for odd perfect numbers, but because its oracle is a version of trial division it gives no complexity advantage over a polynomial-time primality or factoring oracle. The sieve periods are $\mathrm{lcm}(2, \ldots, U)$ (counting) and $U\#$ (OR); every digit oracle is a function of a gcd and hence cannot be injective on a period, while shifted stacking repairs injectivity for a quantity already determined by U. The appendices generalise the encoding to Dirichlet and Boolean divisor transforms, to higher-order series and to prime constellations, where it gives a clean shift-and-NOR oracle for numerical evidence on twin primes. Counting cannot solve the inverse prime-counting problem in $O(\sqrt{N})$ queries. Open questions: injectivity and window size for shifted OR stacking; oracles for $\sigma(k)$ using the digit structure with $U \approx \sqrt{N}$; and oracles exploiting more than black-box access to the sieve.

A Digit Encodings of Dirichlet Convolutions

The constructions of Sections 2–3 are special cases of a general framework: the digit sieve encodes truncated Dirichlet convolutions.

A.1 Weighted digit generating functions

Let $f : \mathbb{N} \to \mathbb{Z}$ be an arithmetic function, $U \geq 2$, and

$$\Sigma_{f,U,b} = \sum_{n=2}^{U} f(n) \frac{b^{-2n}}{1 - b^{-n}} = \sum_{n=2}^{U} f(n) \sum_{j \geq 2} b^{-jn}, \qquad A_f := \sum_{n=2}^{U} |f(n)|.$$

Theorem 25 (Dirichlet digit encoding) *As an identity of absolutely convergent series,* $\Sigma_{f,U,b} = \sum_{k\geq 1} \mathrm{Oracle}_{\mathrm{f},U}(k)\, b^{-k}$ *with*

$$\mathrm{Oracle}_{\mathrm{f},U}(k) = \sum_{\substack{d\mid k \\ 2\leq d\leq U,\ d<k}} f(d).$$

If $f \geq 0$ *and* $b > A_f$*, this is the base-b expansion.*

Proof. The coefficient of b^{-k} receives $f(d)$ from the term $f(d)b^{-jd}$ exactly when $jd = k$, $j \geq 2$, i.e. $d \mid k$ and $d < k$. If $f \geq 0$ and $b > A_f$, every coefficient is a digit in $[0, b-1]$. ■

Signed weights. If f takes negative values, some coefficients are negative and the floor identity of Proposition 4 is not valid as stated. The standard repair is a constant digit offset.

Proposition 26 (Signed weights) *Let* $b > 2A_f$ *and* $\Sigma^{+}_{f,U,b} := \Sigma_{f,U,b} + A_f/(b-1)$*. Then* $\Sigma^{+}_{f,U,b} = \sum_{k\geq 1}(\mathrm{Oracle}_{\mathrm{f},U}(k) + A_f)b^{-k}$ *is a base-b expansion with digits in* $[0, 2A_f]$*, and*

$$\mathrm{Oracle}_{\mathrm{f},U}(k) = \left\lfloor b\left\{b^{k-1}\Sigma^{+}_{f,U,b}\right\}\right\rfloor - A_f.$$

Proof. $A_f/(b-1) = \sum_{k\geq 1} A_f b^{-k}$ adds A_f to every digit. Each digit lies in $[0, 2A_f] \subset [0, b-1]$, and the digits are not eventually $b-1$ because at primes $k > U$ the digit is $A_f < b-1$. Extraction is Proposition 4. ■

A.2 Relation to Dirichlet convolution

With $(f * g)(n) = \sum_{d\mid n} f(d)g(n/d)$ and $\mathbf{1}(n) = 1$, $(f * \mathbf{1})(n) = \sum_{d\mid n} f(d)$. The digit differs from this by truncation.

Corollary 27 *If* $k \geq 2$ *and* $U \geq \lfloor k/2 \rfloor$ *then* $\mathrm{Oracle}_{\mathrm{f},U}(k) = (f * \mathbf{1})(k) - f(1) - f(k)$.

Proof. Every divisor of k other than 1 and k lies in $[2, k/2]$. ■

A.3 Recovering the weights from the digits

The digit at k never contains $f(k)$ itself, so the formula $f = \mu * (f * \mathbf{1})$ cannot be applied to the digits alone: $(f * \mathbf{1})(j) = \mathrm{Oracle}_{\mathrm{f},U}(j) + f(1) + f(j)$ involves the unknown $f(j)$ on the right. The weights can nevertheless be recovered, from digits at the even positions $2k$.

Proposition 28 (Triangular recovery) *For* $2 \leq k \leq U$*,*

$$f(k) = \mathrm{Oracle}_{\mathrm{f},U}(2k) - \sum_{\substack{d\mid 2k \\ 2\leq d<k}} f(d).$$

Hence $f(2), \ldots, f(U)$ *are determined recursively by the digits at positions* $4, 6, \ldots, 2U$*.*

Proof. The divisors d of $2k$ with $2 \leq d \leq U$, $d < 2k$ are those with $d < k$, together with $d = k$ (since $k \leq U$); no divisor of $2k$ lies strictly between k and $2k$. ■

A.4 Examples

For $k \geq 2$ and $U \geq \lfloor k/2 \rfloor$ (so that all proper divisors are included):

1. $f = \mathbf{1}$: $\mathrm{Oracle}_{\mathrm{f},U}(k) = \tau(k) - 2$ (the digit $\mathrm{Oracle}_{\mathrm{count},U}(k)$ of the main text).
2. $f(n) = n$: $\mathrm{Oracle}_{\mathrm{f},U}(k) = \sigma(k) - k - 1$ (the digit $\mathrm{Oracle}_{\mathrm{divisorSum},U}(k)$, used for odd perfect numbers).
3. $f = \mu$: since $\sum_{d|k} \mu(d) = 0$ for $k > 1$, $\mathrm{Oracle}_{\mathrm{f},U}(k) = -\mu(k) - 1$. (The excluded divisor $d = 1$ contributes $\mu(1) = 1$, which is why the digit is $-\mu(k) - 1$ and not $-\mu(k)$.)
4. $f = \varphi$: since $\sum_{d|k} \varphi(d) = k$, $\mathrm{Oracle}_{\mathrm{f},U}(k) = k - 1 - \varphi(k)$.
5. $f = [\, n \text{ prime}\,]$: $\mathrm{Oracle}_{\mathrm{f},U}(k) = \omega(k) - [k \text{ prime}]$, the number of distinct prime factors of k less than k.

The von Mangoldt function Λ satisfies $\sum_{d|k} \Lambda(d) = \log k$, but it is not integer-valued, so it lies outside this framework (a digit cannot hold $\log p$); only a rounded or scaled version could be encoded, with the corresponding error.

A.5 Quantum arithmetic-function oracles

Theorem 29 *Let $f : \mathbb{N} \to \mathbb{Z}$ be computable by a reversible circuit with C_f gates. Then there is a reversible oracle $\mathbf{O}_f : |k\rangle|0\rangle \mapsto |k\rangle|\mathrm{Oracle}_{\mathrm{f},U}(k)\rangle$ with $O(U(\alpha^2 + C_f))$ gates and $O(\alpha + \log A_f)$ qubits. With signed weights the offset of Proposition 26 is applied classically and removed by one subtraction.*

Proof. Loop over $n = 2, \ldots, U$; if $n \mid k$ and $n < k$, add $f(n)$ to an accumulator; uncompute the work registers. This is the circuit of Lemma 7 with the weight $f(n)$ in place of 1. ■

The digit-sieve framework is therefore a general mechanism for oracles of arithmetic functions given as truncated divisor transforms; the original sieve is $f = \mathbf{1}$ and the divisor-sum oracle is $f(n) = n$.

B Boolean Divisor Convolutions and Logical Sieve Encodings

Appendix A generalised the counting sieve by arithmetic functions. Here arithmetic addition is replaced by logical operations, which is natural for the OR sieve of Section 2.1.

B.1 Boolean predicates on divisors

Let $P : \mathbb{N} \to \{0, 1\}$, $U \geq 2$ and $b = 2^r$, $r \geq 1$. Put

$$S_{P,n}(b) = P(n)\,\frac{b^{-2n}}{1 - b^{-n}} = \sum_{j \geq 2} P(n) b^{-jn}, \qquad \Theta_{P,U,b} = \bigvee_{n=2}^{U} S_{P,n}(b),$$

with $\vee$ the bitwise OR (digit-wise, since each non-zero digit occupies the lowest bit of its block).

Theorem 30 (Boolean OR transform) $\Theta_{P,U,b} = \sum_{k\geq 1} \mathrm{Oracle}_{\mathrm{divisorOR},P,U}(k)\, b^{-k}$ *with*

$$\mathrm{Oracle}_{\mathrm{divisorOR},P,U}(k) = \bigvee_{\substack{d\mid k \\ 2\leq d\leq U,\ d<k}} P(d),$$

i.e. the digit is 1 *iff some proper divisor* $d \in [2, U]$ *of* k *satisfies* $P(d)$*. For* $P \equiv 1$ *it equals the OR digit* $\mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k)$ *(without the marker at* $k = 1$*) of Theorem 6.*

Proof. A digit at position k receives a contribution from the series of d iff $d \mid k$, $d < k$ and $P(d) = 1$. Contributions are binary and combined by OR. ■

B.2 AND and XOR transforms

Definition 31 *For* $P : \mathbb{N} \to \{0, 1\}$ *define, over the proper divisors* $d \mid k$ *with* $2 \leq d \leq U$, $d < k$,

$$\mathrm{Oracle}_{\mathrm{divisorOR},P,U}(k) = \bigvee P(d), \tag{100}$$

$$\mathrm{Oracle}_{\mathrm{divisorAND},P,U}(k) = \bigwedge P(d), \tag{101}$$

$$\mathrm{Oracle}_{\mathrm{divisorXOR},P,U}(k) = \bigoplus P(d), \tag{102}$$

with the convention $\bigwedge \varnothing = 1$ *(so a prime satisfies the AND transform vacuously) and* $\bigvee \varnothing = \bigoplus \varnothing = 0$*.*

Respectively these say: some proper divisor satisfies P; every proper divisor satisfies P; the number of proper divisors satisfying P is odd. The XOR transform is the divisor sum over $\mathbf{F}_2$.

The above oracles ,given by equations 100,101,102 , maybe implemented by using standard classical algorithm,or classical circuits, that compute the bitwise operations using bitwise OR, bitwise AND, bitwise XOR on two arbritary rational real numbers represented in a classical computer then using the Bennett reversibility construction to convert the above classical circuits into quantum versions of those circuit. This paragraph also applied to the main body of this paper (outside the appendices) where needs be.

Digit realisations. The OR transform is Theorem 30. The XOR transform is realised by the digit-wise XOR of the series $S_{P,n}(b)$, $b = 2^r$. The AND transform is *not* the digit-wise AND of the series, which would require every $n \leq U$ to divide k; it is realised by complementation, $\mathrm{Oracle}_{\mathrm{divisorAND},P,U}(k) = 1 - \mathrm{Oracle}_{\mathrm{divisorOR},\neg P,U}(k)$. The extraction identity of Proposition 4 applies unchanged in each case (digits in $\{0, 1\}$, not eventually $b - 1$).

B.3 Examples

1. **Compositeness.** $P \equiv 1$ gives $\mathrm{Oracle}_{\mathrm{divisorOR},P,U}(k) = [\exists\, d \mid k,\ 2 \leq d \leq U,\ d < k]$, the compositeness flag for $1 \leq k < (U + 1)^2$ (with the marker at $k = 1$).

2. **Proper prime divisors.** $P(d) = [d \text{ prime}]$ gives $\mathrm{Oracle}_{\mathrm{divisorOR},P,U}(k) = 1$ iff k has a prime divisor $p \leq U$ with $p < k$. (A prime k has no such divisor, because $d = k$ is excluded.)

3. **Non-squarefree numbers.** $P(d) = [d \text{ squarefree}]$ (equivalently $\mu(d) \bmod 2$). For $k \geq 2$ and $U \geq \lfloor k/2 \rfloor$ the number of squarefree proper divisors in $[2, k)$ is $2^{\omega(k)} - 1$ if k is not squarefree and $2^{\omega(k)} - 2$ if it is, so $\mathrm{Oracle}_{\mathrm{divisorXOR},P,U}(k) = 1$ iff k is not squarefree.

B.4 Logical oracles

Corollary 32 *For each of the three transforms there is a reversible oracle* $|k\rangle|0\rangle \mapsto |k\rangle|\mathrm{Oracle}_{\cdot,P,U}(k)\rangle$ *with* $O(U(\alpha^2 + C_P))$ *gates and* $O(\alpha)$ *ancillary qubits, where* C_P *is the cost of evaluating* $P(d)$ *(a constant for the three examples above).*

Proof. Evaluate $[d \mid k \wedge d < k]$ for each $d \in \{2, \ldots, U\}$, combine $P(d)$ into a one-bit accumulator by OR, AND or XOR, copy out and uncompute. ■

B.5 An $\mathbf{F}_2$ identity: OR in terms of XOR and AND

The XOR transform of this appendix has non-trivial uses one such example use is given as follows: together with AND it expresses OR.

Proposition 33 *For bits* $x_1, \ldots, x_m$ *(*$m \geq 1$*),*

$$x_1 \vee \cdots \vee x_m = \bigoplus_{\varnothing \neq S \subseteq \{1,\ldots,m\}} \bigwedge_{i \in S} x_i.$$

In particular $A \vee B = (A \oplus B) \oplus (A \wedge B)$*.*

Proof. If all $x_i = 0$ both sides are 0. If exactly $m' \geq 1$ of the x_i equal 1, the non-empty subsets S all of whose members equal 1 number $2^{m'} - 1$, which is odd, so the XOR of the corresponding terms is 1. ■

Corollary 34 *Let* $D(k) = \{d \mid k :\ 2 \leq d \leq U,\ d < k\}$*. Then*

$$\mathrm{Oracle}_{\mathrm{divisorOR},P,U}(k) = \bigoplus_{\varnothing \neq S \subseteq D(k)} \bigwedge_{d \in S} P(d).$$

If $P = P_1 \vee P_2$*, then, writing* $A = \mathrm{Oracle}_{\mathrm{divisorOR},P_1,U}(k)$ *and* $B = \mathrm{Oracle}_{\mathrm{divisorOR},P_2,U}(k)$*,*

$$\mathrm{Oracle}_{\mathrm{divisorOR},P,U}(k) = \big(A \oplus B\big) \oplus \big(A \wedge B\big).$$

Proof. The first formula is Proposition 33 applied to the bits $P(d)$, $d \in D(k)$. For the second, $A \vee B$ is the OR over $D(k)$ of $P_1(d) \vee P_2(d)$, since OR is associative and commutative. ■

Example. Let $P_1(d) = [\,2 \mid d\,]$, $P_2(d) = [\,3 \mid d\,]$ and $k = 6$, $U \geq 3$, so $D(6) = \{2, 3\}$. Then $A = 1$ (from $d = 2$), $B = 1$ (from $d = 3$), and $A \vee B = 1 = (1 \oplus 1) \oplus (1 \wedge 1) = 0 \oplus 1$. For $k = 12$, $D(12) = \{2, 3, 4, 6\}$ with $P \equiv 1$, the sum has $2^4 - 1 = 15$ non-empty subsets, all with AND equal to 1, and 15 is odd, so the XOR is 1, the OR.

Remark. The expansion in Corollary 34 has $2^{|D(k)|} - 1$ terms and is no shortcut for a circuit, which computes the OR directly with one accumulator bit (Lemma 7). Its value is structural: the OR transform lies in the $\mathbf{F}_2$-algebra generated by AND and XOR transforms, and the two-source form combines two sieve digits with a constant number of gates. In base 2^r the same identity holds digit-wise for the real numbers $\Theta_{P_1,U,b}$ and $\Theta_{P_2,U,b}$.

B.6 Relation to Appendix A

Appendix A studies $(f * \mathbf{1})(n) = \sum_{d|n} f(d)$; the present appendix studies $\bigvee_{d|n} P(d)$, $\bigwedge_{d|n} P(d)$ and $\bigoplus_{d|n} P(d)$, with $(+, \times)$ replaced by logical operations on the divisor lattice. The OR sieve of the main text is the simplest instance.

C Higher-Order Divisor Transforms

Appendix A used $1/(1-x) = \sum_{r\geq 0} x^r$. The generalisation

$$\frac{1}{(1-x)^m} = \sum_{r\geq 0} \binom{r+m-1}{m-1} x^r$$

produces polynomial rather than constant coefficients.

C.1 Definition

For $m \geq 1$ and $f : \mathbb{N} \to \mathbb{Z}_{\geq 0}$ put

$$\Sigma^{(m)}_{f,U,b} = \sum_{n=2}^{U} f(n) \frac{b^{-2n}}{(1-b^{-n})^m}, \qquad A^{(m)}(j) = \sum_{n=2}^{U} f(n) \binom{\lceil j/n \rceil + m - 3}{m-1}.$$

For $m \geq 2$ the coefficients grow with the position, so carry-freeness must be ensured for positions $k \leq N$ together with the influence of the infinite tail.

Theorem 35 *The series equals* $\sum_{k\geq 1} \mathrm{Oracle}^{(m)}_{f,U}(k) b^{-k}$ *with*

$$\mathrm{Oracle}^{(m)}_{f,U}(k) = \sum_{\substack{d|k \\ 2\leq d\leq U,\ d<k}} f(d) \binom{k/d + m - 3}{m-1}.$$

Assume $b > A^{(m)}(N)$ *and* $\sum_{i\geq 1} A^{(m)}(N+i)\, b^{-i} < 1$ *(the latter holds for all sufficiently large* b*, since* $A^{(m)}$ *grows polynomially). Then for every* $k \leq N$ *the value* $\mathrm{Oracle}^{(m)}_{f,U}(k)$ *is the digit of* b^{-k} *in the base-*b *expansion.*

Proof. By the negative binomial expansion, $b^{-2d}(1-b^{-d})^{-m} = \sum_{r\geq 0} \binom{r+m-1}{m-1} b^{-(r+2)d}$. Setting $k = (r+2)d$ gives $r = k/d - 2$ and the coefficient $\binom{k/d+m-3}{m-1}$. For the digit claim: for $j \leq N$ the coefficients satisfy $\mathrm{Oracle}^{(m)}_{f,U}(j) \leq A^{(m)}(N) < b$; the tail $T_N = \sum_{i\geq 1} \mathrm{Oracle}^{(m)}_{f,U}(N+i) b^{-i}$ is < 1 by hypothesis. For $k \leq N$ the tail after position k is at most $\sum_{i=1}^{N-k} (b-1) b^{-i} + b^{-(N-k)} T_N < 1$, so no carry reaches position k. ∎

C.2 Examples

For $m = 1$ the digit is $\sum_{d|k,\, 2\leq d<k} f(d)$, as in Appendix A. For $m = 2$ it is $\sum f(d)(k/d - 1)$, a cofactor-weighted divisor transform; for $m = 3$ it is $\sum f(d)\, \frac{(k/d)(k/d-1)}{2}$, with quadratic cofactor weights. For example, $f \equiv 1$, $m = 2$ and $U \geq \lfloor k/2 \rfloor$ gives $\sum_{e|k,\, 2\leq e<k} (e-1) = \sigma(k) - k - 1 - (\tau(k) - 2)$.

C.3 Quantum oracle

Under the hypotheses of Theorem 35, $\mathrm{Oracle}^{(m)}_{f,U}(k) = \lfloor b\{b^{k-1}\Sigma^{(m)}_{f,U,b}\}\rfloor$ for $k \le N$, and there is a reversible oracle $\mathbf{O}^{(m)}_f : |k\rangle|0\rangle \mapsto |k\rangle|\mathrm{Oracle}^{(m)}_{f,U}(k)\rangle$ built from modular arithmetic and accumulation (the quotient k/d is read off the same division that tests $d \mid k$).

D Higher-Order Boolean Divisor Transforms

Appendix B introduced OR, AND and XOR transforms. The generating-function expansion of Appendix C extends to this setting.

D.1 Higher-order OR encoding

For $P : \mathbb{N} \to \{0,1\}$ let $\Theta^{(m)}_{P,U,b} = \bigvee_{n=2}^{U} P(n)\, b^{-2n}(1-b^{-n})^{-m}$, defined digit-wise: a position is 1 if some term has non-zero coefficient there.

Theorem 36 *$\Theta^{(m)}_{P,U,b}$ has digit $\bigvee_{d\mid k,\, 2\le d<k,\, d\le U} P(d)$ at position k. Hence every higher-order OR transform equals the ordinary OR transform of Appendix B.*

Proof. The coefficients $\binom{r+m-1}{m-1} \ge 1$ are non-zero at exactly the positions $(r+2)d$ for $r \ge 0$, the same positions as for $m = 1$. Occupancy alone determines the OR output. ■

D.2 Higher-order XOR transforms

The XOR transform is not invariant, because multiplicities are reduced mod 2:

$$\mathrm{Oracle}^{(m)}_{\mathrm{XOR},P,U}(k) = \bigoplus_{\substack{d\mid k\\ 2\le d<k,\ d\le U}} P(d)\Big(\binom{k/d+m-3}{m-1} \bmod 2\Big).$$

By Lucas' theorem the parities of the binomial coefficients follow Sierpiński-type patterns, which control which divisor contributions survive. The AND transform is obtained by complementation as in Appendix B.

D.3 Quantum logical oracles

Every higher-order transform yields a reversible oracle $\mathbf{O}^{(m)}_{\vee}, \mathbf{O}^{(m)}_{\oplus}, \mathbf{O}^{(m)}_{\wedge}$ with the same $O(U\alpha^2)$-gate, $O(\alpha)$-qubit scaling (for fixed m), giving a logical hierarchy parallel to the arithmetic hierarchy of Appendix C.

E Prime Constellations and Numerical Evidence for Twin Primes

This appendix applies the OR digit to joint predicates such as "k and $k+2$ are both prime". We emphasise at the outset: a finite computation cannot prove that there are infinitely many twin primes. The procedure below produces numerical evidence only. This is the first concrete analytic operator based algorithm for generic prime constellations (there does exist a twin prime quantum algorithm which is a special case see [12]) that embeds a prime constellation

structure into a generalised Lambert Transform. Our prime constellations verification which is a quadratic faster than the best known classical algorithm for finding arbitrary prime constellations; because on an arbitrarty sized interval $[\eta_1, \eta_2]$ the best classical algorithm takes $O(\log^2(\eta_2 - \eta_1) \log\log(\eta_2 - \eta_1))$ time complexity

E.1 Shift-and-NOR oracle

By Theorem 6 the series $\Theta_{U,b}$ is a *compositeness* stream: its digit at k is 1 iff $k = 1$ or k has a proper divisor in $[2, U]$. (Weighting the series by $[\,n \text{ prime}\,]$ also gives a compositeness stream, and needs the primes in advance; the choice $P \equiv 1$ does not.) Multiplying by b^c moves the digit at position $k + c$ to position k, so the real number $b^c\Theta_{U,b}$ has, at position k, the digit $\mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k + c)$. Primality is the complement.

Definition 37 *For an offset set $C = \{0 = c_1 < c_2 < \cdots < c_t\}$ and $k \geq 1$ with $k + c_t < (U+1)^2$,*

$$\mathrm{Oracle}_{\mathrm{constellation},U,C}(k) = \bigwedge_{j=1}^{t} \Big(1 - \mathrm{Oracle}_{\mathrm{bitwiseOR},U}(k + c_j)\Big),$$

which equals 1 iff $k + c_1, \ldots, k + c_t$ are all prime. For $C = \{0, 2\}$ this is the twin-prime predicate.

The conjunction of the compositeness flags themselves would detect the opposite event (all composite); the complement, a NOR, is what is needed. Fixed gaps $2m$ use $C = \{0, 2m\}$; Dickson's conjecture concerns admissible C. A shift by a constant is a constant addition on the index; if a shift is held in a register, b^c is formed by fast binary exponentiation (Remark 8).

Cost. A circuit evaluates the t flags one after another on a single index register (add c_j, compute the flag with the circuit of Lemma 7, store, uncompute), then applies one multi-controlled gate on t bits. The cost is $t \cdot O(U\alpha^2)$ gates and $O(\alpha + t)$ qubits. This equals the cost of the multi-register construction; the shift-and-NOR description is a cleaner algebraic formulation, not an asymptotic saving. With a deterministic polynomial-time primality oracle the per-flag cost is $\mathrm{poly}(\alpha)$ and no sieve bound U is needed.

E.2 Bounded existence search

Theorem 38 (Existence of a constellation in a window) *Let $W = [H, H + N)$ and $U = \lceil\sqrt{H + N + c_t}\,\rceil$, and let M be the number of $k \in W$ with $\mathrm{Oracle}_{\mathrm{constellation},U,C}(k) = 1$. Amplitude amplification with an unknown number of solutions (the schedule of Algorithm 2) decides, with one-sided error and classical verification of every output, whether $M \geq 1$ using $O(\sqrt{N})$ queries in the worst case and expected $O(\sqrt{N/M})$ queries when $M \geq 1$. Each query costs $t \cdot O(U\alpha^2)$ gates with the digit oracle, or $t \cdot \mathrm{poly}(\alpha)$ with a polynomial-time primality oracle.*

Proof. As Theorem 14, with the predicate $\mathrm{Oracle}_{\mathrm{constellation},U,C}$ in place of $\mathrm{Oracle}_{\mathrm{oddPerfect},U}$. The sieve bound depends on the *top* of the window, $H + N$, not on N. ∎

Under the Hardy–Littlewood prime t-tuple heuristic [9], $M \approx \mathfrak{S}_C\, N/\ln^t H$ with a constant $\mathfrak{S}_C > 0$ for admissible C (for twins $\mathfrak{S} = 2C_2 \approx 1.32$), so the expected number of queries is $O(\ln^{t/2} H)$, for twins $O(\ln H)$. For comparison, a classical random sampler needs $O(\ln^t H)$ expected primality tests ($O(\ln^2 H)$ for twins), and exhaustive scanning of the window needs N. The quantum gain is therefore quadratic in the *number of samples*; it is not a statement about proving existence, since the heuristic $M \geq 1$ for windows of length $\gg \ln^t H$ is itself conjectural.

E.3 An evidence-gathering procedure

Algorithm 4 (numerical evidence for twin primes, $C = \{0, 2\}$). For a sequence of heights $H_1 < H_2 < \cdots$ and window lengths $N_i \geq K \ln^2 H_i$:

1. **Existence.** Run the search of Theorem 38 on $[H_i, H_i + N_i)$; if a pair is output, verify both members classically and record it.
2. **Density.** Run quantum counting [5] with $P = O(\sqrt{N_i/M}/\varepsilon) = O(\ln H_i/\varepsilon)$ queries to obtain $\tilde{M}_i$ with relative error ε (the argument of Theorem 9 with M in place of $\pi(N-1)$).
3. **Comparison.** Compare $\tilde{M}_i$ with the Hardy–Littlewood prediction $2C_2 \int_{H_i}^{H_i+N_i} dt/\ln^2 t$ and tabulate.

With a polynomial-time primality oracle the total cost per height is $\mathrm{poly}(\alpha) \ln H_i/\varepsilon$ gates, against $\ln^2 H_i/\varepsilon^2$ samples for classical Monte Carlo density estimation. An *exact* count of twin primes in a window still needs $\Omega(\sqrt{M(N-M)})$ queries (Theorem 23) and is not accelerated. The output is a table of verified twin primes and density estimates at increasing heights: evidence in support of the heuristic, never a proof of infinitude.

E.4 Comparison with classical methods

1. *Query model.* For M solutions among N candidates a classical randomised algorithm needs $\Theta(N/M)$ oracle queries to find one, a quantum algorithm $\Theta(\sqrt{N/M})$ [3]. To estimate the density $p = M/N$ to relative error ε a classical sampler needs $\Theta(1/(p\varepsilon^2))$ queries and quantum counting $\Theta(1/(\sqrt{p}\,\varepsilon))$ [5, 15]. The speed-up is therefore exactly quadratic in the number of queries for *every* M; the density only changes the common factor. For twins, $p \approx 2C_2/\ln^2 H$ gives $\ln H$ versus $\ln^2 H$ queries for existence, and $\ln H/\varepsilon$ versus $\ln^2 H/\varepsilon^2$ for the density. In the worst case ($M \leq 1$) it is $\sqrt{N}$ versus N.
2. *Classical baseline.* For evidence at height H the natural classical method samples candidates, pre-filtered by a wheel or small-prime sieve, with a polynomial-time primality test. The same pre-filter can be built into the quantum search domain (residues admissible modulo a primorial), so the comparison is in query count at equal oracle cost.
3. *Exact counting.* Classical sieving of a window costs $\tilde{O}(N)$ and we know of no sublinear classical exact twin-prime count; the quantum exact count needs $\Omega(\sqrt{M(N-M)})$ queries (Theorem 23). There is no speed-up for exact counting.
4. *Guarantees.* The quantum existence search carries an unconditional worst-case bound: $O(\sqrt{N})$ queries decide, with one-sided error, whether a window of length N contains a constellation, and the expected number of queries is $O(\sqrt{N/M})$ when $M \geq 1$ solutions exist; this follows from amplitude amplification with no number-theoretic hypothesis, and is optimal in the query model up to a constant [2]. The classical counterpart in the same model is also unconditional: $\Theta(N)$ queries in the worst case and $\Theta(N/M)$ in expectation, so the separation is a provable quadratic one. What is heuristic on the classical side is the expected cost of *finding* a pair by sampling or by record-hunting searches: the claim that about $\ln^2 H$ candidates suffice rests on the Hardy–Littlewood density heuristic, and a structured search is not guaranteed to reach a pair. The individual steps of those methods (sieving, primality testing) have proven running times; what they lack is a proven bound on the number of candidates that must be examined. The same heuristic enters the quantum expected cost $O(\ln^{t/2} H)$ whenever one substitutes the predicted density for M.

5. *Not compared.* Specialised record searches for large twin primes use structured candidates and sieving followed by dedicated primality tests, exploiting structure unavailable to a black-box oracle; analytic results prove infinitely many pairs of primes with bounded gap [22] but not twin primes. No search, classical or quantum, can prove the twin prime conjecture. A quantum advantage also presupposes a fault-tolerant quantum computer running the primality circuit at height H.

E.5 Prime constellations and Dickson's conjecture

Admissibility and local obstructions. A finite set of shifts $C = \{c_1, \dots, c_t\}$ is *admissible* if there is no prime p for which the residues $c_1, \dots, c_t$ cover every residue class modulo p. If such a p existed, then for every integer n one of the numbers $n + c_j$ would be divisible by p, so $n + c_1, \dots, n + c_t$ could all be prime only when that member equals p itself, which happens for at most finitely many n. Only primes $p \le t$ can cover all residues, so admissibility is a finite check. For example $\{0, 2\}$ is admissible, while $\{0, 2, 4\}$ is not, since it covers all residues modulo 3 (only $3, 5, 7$ is prime). Dickson's conjecture, in the prime t-tuple form used here, states that in the absence of such a local obstruction every finite admissible constellation occurs for infinitely many n [7]; the singular series $\mathfrak{S}_C$ in the Hardy-Littlewood heuristic is positive exactly when C is admissible.

For any finite admissible C the same construction applies with $t = |C|$ shifts: Theorem 38 holds with the bound $O(\sqrt{N})$ queries (constant t) and $O(\ln^{t/2} H)$ expected queries under the prime t-tuple heuristic. As for twins, a window search yields evidence, not proof.

F A Generalized Lambert Transform Framework

The classical Lambert series $\sum_n f(n)\, x^n/(1 - x^n) = \sum_k \big(\sum_{d|k} f(d)\big) x^k$ already expresses a divisor sum as a coefficient. The digit sieves of the main text and of Appendices A–D are Lambert series with a modified numerator, and can be unified by a kernel.

Definition 39 *Let $g(z) = \sum_{r \ge 0} a_r z^r$ be a formal power series with integer coefficients (the* kernel*), $A(m) := \sum_{r=0}^{m} a_r$, $f : \mathbb{N} \to \mathbb{Z}$ and $U \ge 2$. The generalized Lambert transform is*

$$S_{f,g,U}(x) = \sum_{n=2}^{U} f(n)\, \frac{g(x^n)}{1 - x^n}.$$

Theorem 40 *$S_{f,g,U}(x) = \sum_{k \ge 0} c_{f,g,U}(k)\, x^k$, where for $k \ge 1$*

$$c_{f,g,U}(k) = \sum_{\substack{d \mid k \\ 2 \le d \le U}} f(d)\, A(k/d).$$

Proof. $g(x^n)/(1 - x^n) = \sum_r a_r x^{nr} \sum_{s \ge 0} x^{ns} = \sum_{m \ge 0} A(m) x^{nm}$. Put $k = nm$. ∎

With $x = 1/b$ the coefficient $c_{f,g,U}(k)$ is the digit at position k, and the truncation $d \le U$ is the only place the sieve bound enters.

Kernel $g(z)$	$A(m)$	Transform
z	1 for $m \ge 1$	full divisor sum $\sum_{d \mid k} f(d)$ (classical Lambert series)
z^2	$[\, m \ge 2 \,]$	proper-divisor transform (main text, Appendix A)
z^j	$[\, m \ge j \,]$	divisors with $d \le k/j$
$z^2(1 - z)^{-(m-1)}$	$\binom{m'+m-3}{m-1}$ at m'	higher-order transform (Appendix C)

For the higher-order kernel, $g(z)/(1-z) = z^2/(1-z)^m$, which is why the exponent of $(1-z)$ in the kernel is $-(m-1)$ and not $m-1$. For $m=1$ the kernel is z^2.

Digit extraction and oracles. If $f \geq 0$, the a_r are non-negative integers, and the base satisfies the carry and tail conditions of Theorem 35 with $A(\cdot)$ in place of the binomial coefficient, then $c_{f,g,U}(k) = \lfloor b\{b^{k-1}S_{f,g,U}(1/b)\}\rfloor$ for $k \leq N$, and the reversible oracle has $O(U(\alpha^2 + C_f + C_A))$ gates, where C_A is the cost of evaluating A. Signed weights use the offset of Proposition 26.

Boolean semiring. Replace $(+,\times)$ by $(\vee,\wedge)$ for f and $a_r \in \{0,1\}$:

$$c^{\vee}_{f,g,U}(k) = \bigvee_{\substack{d|k \\ 2\leq d\leq U}} f(d) \wedge A_{\vee}(k/d), \qquad A_{\vee}(m) = \bigvee_{r\leq m} a_r = [\, m \geq r_{\min}\,],$$

with $r_{\min} = \min\{r : a_r = 1\}$. Hence every Boolean OR transform with a non-negative kernel depends on the kernel only through $r_{\min}$, which explains the collapse of the higher-order OR transforms of Appendix D to the ordinary one ($r_{\min} = 2$ for $z^2(1-z)^{-(m-1)}$). The XOR version uses $A_{\oplus}(m) = \bigoplus_{r\leq m} a_r$, and for the higher-order kernel $A_{\oplus}(j) = \binom{j+m-3}{m-1} \bmod 2$, the Lucas-theorem patterns of Appendix D.

Remark 41 *Appendices A,B,C,D are thus the kernels z^2 and $z^2(1-z)^{-(m-1)}$ over the integers and over the Boolean semiring. This is a reorganisation of existing results; no new complexity claim is made.*

G The Dirichlet Ring Structure of the Kernel Transforms

This appendix records how the generalized Lambert transforms of Appendix F sit inside the standard ring of arithmetic functions under Dirichlet convolution. The facts are standard and are stated for completeness; no new complexity claim is made.

G.1 The ring and the operators

Let $\mathcal{R}$ be the set of functions $\mathbb{N} = \{1,2,\dots\} \to \mathbb{Z}$ with pointwise addition and Dirichlet convolution $(u * v)(n) = \sum_{d|n} u(d)v(n/d)$. It is a commutative ring with identity $\delta(n) = [\, n = 1\,]$. Since $(u*v)(1) = u(1)v(1)$, the set $\mathcal{I} = \{f \in \mathcal{R} : f(1) = 0\}$ is an ideal.

Let $g(z) = \sum_{r\geq 0} a_r z^r$ be a formal power series with integer coefficients and $A(m) = \sum_{r=0}^{m} a_r$. Regard A, restricted to $m \geq 1$, as an element of $\mathcal{R}$ (the value $A(0) = a_0$ plays no role below). Define, for $A \in \mathcal{R}$,

$$(T_A f)(k) = \sum_{d|k} f(d)\, A(k/d) = (f * A)(k).$$

Proposition 42 (Dirichlet convolution representation) *1. The untruncated generalized Lambert transform of Theorem 40 ($U = \infty$) with weights $f \in \mathcal{I}$ has coefficients $c_{f,g}(k) = (T_A f)(k) = (f * A)(k)$. (The sum over $d \geq 2$ in the main text is the sum over all d for $f \in \mathcal{I}$, since $f(1) = 0$.)*

2. *$T_A + T_B = T_{A+B}$, $T_A \circ T_B = T_{A*B}$ and $T_\delta = \mathrm{id}$. Hence $A \mapsto T_A$ is a ring isomorphism from $(\mathcal{R}, +, *)$ onto the ring of operators $\{T_A\}$, which is therefore commutative and isomorphic to the standard Dirichlet ring. Every sequence $A(1), A(2), \dots$ arises from a kernel, namely $a_r = A(r) - A(r-1)$ for $r \geq 1$.*

3. *In the basis of unit functions $e_1, e_2, \ldots$, T_A has the matrix $(M_A)_{k,d} = A(k/d)\,[\,d \mid k\,]$, which is lower triangular because $d \mid k$ implies $d \le k$; and $M_A M_B = M_{A*B}$.*

Proof. (1) is Theorem 40 with the truncation removed. (2) $T_A(T_B f) = (f * B) * A = f * (A * B)$ by associativity and commutativity of $*$; additivity and $T_\delta = \mathrm{id}$ are immediate; injectivity of $A \mapsto T_A$ follows from $T_A(\delta) = A$. (3) is the matrix form of $f \mapsto f * A$. ■

Consequently the operators T_A are *multiplication in the Dirichlet ring*, and a transform is a module homomorphism over $\mathcal{R}$ in the trivial sense that $T_A(h * f) = h * T_A f$.

G.2 Specific kernels

- **Proper-divisor sieve kernel.** For $g(z) = z^2$, $A(m) = [\,m \ge 2\,] = \zeta - \delta$, where $\zeta \equiv 1$. Hence $T_A f = f * \zeta - f$: convolution with ζ followed by subtraction of f, which removes the term $d = k$. For $f \in \mathcal{I}$ the term $d = 1$ is absent as well, and $(T_A f)(k) = \sum_{d \mid k,\, 2 \le d < k} f(d)$, the digit of Theorem 25.

- **Higher-order transforms.** The rational kernel $g(z) = z^2(1-z)^{-(m-1)}$ gives $g(z)/(1-z) = z^2/(1-z)^m$ and $A_m(j) = \binom{j+m-3}{m-1}$ for $j \ge 2$, with $A_m(1) = 0$. These sequences are related by partial sums, $A_{m+1}(j) = \sum_{i \le j} A_m(i)$ (the hockey-stick identity); this is an additive operation, not a Dirichlet one.

G.3 Limitations

- **Truncation.** Restricting to $d \le U$ breaks closure under composition: the composite of two truncated transforms sums over intermediate divisors $d \le U$, whereas $T^{(U)}_{A*B}$ sums over all of them. The ring statement holds only in the untruncated limit.

- **Boolean version.** The Boolean Dirichlet convolution $(u \star v)(n) = \bigvee_{d \mid n} u(d) \wedge v(n/d)$ makes $\{0,1\}$-valued functions a commutative semiring (idempotent, with identity δ), not a ring, because $\vee$ has no inverses. The composition law $T^{\vee}_A \circ T^{\vee}_B = T^{\vee}_{A \star B}$ still holds by associativity, but the kernels z^r, for which $A_r(m) = [\,m \ge r\,]$, do not compose multiplicatively: $(A_r \star A_s)(n) = [\,\exists\, d \mid n : \ r \le d \le n/s\,]$, which differs from A_{rs} (for example $(A_2 \star A_2)(5) = 0$ but $A_4(5) = 1$).

Remark 43 *In the language of incidence algebras, the operators T_A are the commutative subalgebra of matrices supported on divisibility pairs whose entries depend only on the quotient k/d. the structure recorded here is therefore is commutative and may be part of a bigger algebraic structure . Whether a digit encoding can realise weights that depend on the position k and not only on k/d, which would reaches a larger algebra, is open.*

References

[1] Locate ref

[2] C. H. Bennett, E. Bernstein, G. Brassard, U. Vazirani, Strengths and weaknesses of quantum computing, *SIAM J. Comput.* 26 (1997), 1510–1523.

[3] M. Boyer, G. Brassard, P. Høyer, A. Tapp, Tight bounds on quantum searching, *Fortschr. Phys.* 46 (1998), 493–505.

[4] G. Brassard, P. Høyer, A. Tapp, Quantum counting, *ICALP 1998*, LNCS 1443, 820–831.

[5] G. Brassard, P. Høyer, M. Mosca, A. Tapp, Quantum amplitude amplification and estimation, *Contemp. Math.* 305 (2002), 53–74.

[6] Locate ref

[7] Locate ref

[8] L. K. Grover, A fast quantum mechanical algorithm for database search, *STOC 1996*, 212–219.

[9] Hardy et al

[10] Locate ref

[11] .Locate ref

[12] Locate ref

[13] .Locate ref

[14] Locate ref

[15] A. Nayak, F. Wu, The quantum query complexity of approximating the median and related statistics, *STOC 1999*, 384–393.

[16] Locate ref

[17] A quantum algorithm for Zig-Zag Functions and a Grover Based Algorithm for Golden Mean, draft manuscript,2026

[18] Locate ref

[19] Locate ref

[20] P. W. Shor, Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, *SIAM J. Comput.* 26 (1997), 1484–1509.

[21] Locate ref

[22] Locate ref.